**Workflow B: AI-Driven Worker Proximity Matching & Dispatch**

1. **Spatial Filtering:** The system identifies all verified workers belonging to the requested trade with `is_available = TRUE`.
2. **Haversine Distance Computation:** The GIS module computes distance $d$ in kilometers between job coordinates and worker coordinates: $$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta \lambda}{2}\right)}\right)$$
3. **Ranking Matrix:** AIService combines Haversine proximity, average customer ratings, and current active jobs to rank the top 3 closest artisans.
4. **Claiming:** In Emergency mode, the job is pushed via WebSockets to workers within 7.5 km. The first worker to tap "Claim" locks the assignment.

**Workflow C: Arrival OTP Verification & Field Execution**

1. **In-Transit:** The artisan marks status as "In-Transit", prompting the customer app to display an arrival map and the generated secret OTP.
2. **Physical Gate Check:** The technician arrives at the home and requests the OTP.
3. **Validation:** The technician enters the OTP on their mobile interface ( `/worker/service-request/<id>/verify-arrival-otp` ). The backend validates the entry. On match, status shifts to `in_progress`, unlocking the task timer and safety checklists.
4. **Safety & Completion:** The worker checks off mandatory safety items (e.g., "Main power isolated"), completes physical repairs, takes a photo of the completed work, and clicks "Submit Work".

**Workflow D: Digital Billing, E-Signatures & Statutory Fair Wage Split**

1. **Invoice Computation:** Server computes itemized charges: Labor + Consumables + 18% CGST/SGST.
2. **Customer Sign-off:** The customer reviews the work, navigates to the invoice screen, and draws their physical signature on an HTML5 `<canvas>` element.
3. **Execution of Payment:** The customer settles via Cash or UPI. The server's ledger splits the incoming amount:
    - **88%** allocated to `payments.worker_amount` (Direct living wage).
    - **8%** allocated to `payments.contractor_amount` (Society operations).
    - **4%** allocated to `payments.welfare_amount` and automatically appended as a credit in `welfare_transactions` (*Suraksha Nidhi* passbook).
4. **PDF Generation:** ReportLab dynamically compiles a branded PDF invoice embedding the customer signature, GST breakup, and society registration seal for download.

```
%% Step 3: Physical Arrival Verification
Worker->>Customer: Physically Arrives at Doorstep
Customer-->>Worker: Provides 4-Digit OTP (e.g., "4829")
Worker->>Flask: POST /worker/service-request/<id>/verify-arrival-otp
Flask->>DB: Match OTP -> UPDATE Status: 'in_progress'

%% Step 4: Completion & Invoicing
Worker->>Flask: Complete Safety Checklist, Upload Photo & Finish
Flask->>DB: UPDATE Status: 'completed'
Flask->>Customer: Display Invoice (Labor + Materials + 18% GST)

%% Step 5: Sign & Split Payment
Customer->>WebApp: Draw Canvas E-Signature & Pay (UPI / Cash)
WebApp->>Flask: POST /customer/invoice/sign & pay
Flask->>DB: INSERT into payments (Split: 88% Worker, 8% Society, 4% Welfare)
Flask->>DB: INSERT into welfare_transactions (Credit +4% to Suraksha Nidhi)
Flask-->>Customer: Download Stamped PDF Invoice (ReportLab)
```

## Detailed Workflow Walkthroughs

### Workflow A: Customer Service Booking & Geolocation Resolution

1. **Initiation:** Customer logs in, navigates to `/customer/book-service`, and chooses a trade (e.g., *Electrician*).
2. **Data Capture:** User enters problem details, optionally records a 15-second voice note via the browser's MediaRecorder, and clicks "Use Current Location".
3. **Geocoding:** Browser retrieves HTML5 GPS coordinates (`latitude`, `longitude`); client invokes `/customer/api/geocode` proxy to fetch formatted street address.
4. **State Machine Creation:** Server generates a random 4-digit numeric OTP, inserts a new `service_requests` record with status `pending`, and immediately opens a WebSockets room `booking_<id>` for tracking.

## 5. System Workflow Details (For Design Diagrams)

These workflow steps are structured so they can be translated directly into UML Use Case, Activity, and Sequence Diagrams.

mermaid

```
sequenceDiagram
    autonumber
    actor Customer
    participant WebApp as Web / Mobile App
    participant Flask as Flask Core Engine
    participant AI_GIS as AI & GIS Services
    actor Worker
    actor Admin as Society Admin
    participant DB as MySQL Database

    %% Step 1: Booking
    Customer->>WebApp: Fill Booking Form (Trade, Urgency, Voice Note, GPS)
    WebApp->>Flask: POST /customer/book-service/submit
    Flask->>AI_GIS: Geocode Address & Generate 4-Digit OTP
    Flask->>DB: INSERT into service_requests (Status: 'pending')
    Flask-->>Customer: Display Booking Confirmation & Arrival OTP

    %% Step 2: Matching / Dispatch
    alt Emergency / Auto-Dispatch
        Flask->>AI_GIS: Calculate Haversine Proximity & Rank Workers
        Flask->>Worker: WebSocket Push Notification (Urgent Broadcast)
        Worker->>Flask: POST /worker/service-request/<id>/claim
    else Manual Admin Dispatch
        Admin->>Flask: Assign Worker from Recommended AI List
    end
    Flask->>DB: UPDATE service_requests (Status: 'assigned', 'in_transit')
```

### 4.2 Entity-Relationship (ER) Architecture

The entity relationships map directly to standard relational integrity constraints:

mermaid

```
erDiagram
    users ||--o| contractors : "registers as (1:1)"
    users ||--o| customers : "registers as (1:1)"
    users ||--o| workers : "registers as (1:1)"

    contractors ||--|{ workers : "employs / affiliates (1:N)"
    contractors ||--|{ contractor_documents : "uploads (1:N)"
    contractors ||--|{ contractor_reviews : "receives (1:N)"

    customers ||--|{ service_requests : "places (1:N)"
    contractors ||--|{ service_requests : "manages (1:N)"
    workers ||--o{ service_requests : "assigned to (1:N)"

    service_requests ||--o| quotations : "estimated by (1:1)"
    service_requests ||--o| payments : "settled via (1:1)"
    service_requests ||--|{ attendance : "logs (1:N)"
    service_requests ||--o| contractor_reviews : "generates (1:1)"

    payments ||--o{ welfare_transactions : "auto-credits cess to (1:1)"
    workers ||--|{ welfare_transactions : "owns passbook (1:N)"
    workers ||--|{ worker_certifications : "holds (1:N)"
    workers ||--|{ attendance : "records (1:N)"
```

- `users` **to Sub-Entities:** 1-to-1 relationship between `users` and `contractors`, `customers`, or `workers` using `user_id` as a unique foreign key.
- `contractors` **to** `workers`**:** 1-to-Many (`contractor_id`). Each Labour Cooperative Society manages multiple affiliated artisans.
- `customers` **to** `service_requests`**:** 1-to-Many (`customer_id`). A client can request multiple household repairs over time.
- `service_requests` **to** `payments`**:** 1-to-1 (`service_request_id`). Every completed booking generates a single statutory split payment record.
- `payments` **to** `welfare_transactions`**:** 1-to-1 (`payment_id`). Slices 4% of the payment directly into the designated artisan's *Suraksha Nidhi* passbook record.

8. `welfare_transactions` **(Worker Suraksha Nidhi Passbook):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `worker_id` (FK $\rightarrow$ `workers.id`)
    - `payment_id` (FK $\rightarrow$ `payments.id`, NULLABLE)
    - `transaction_type` (ENUM: `'credit_cess'`, `'pmsby_premium'`, `'pmjjby_premium'`, `'medical_grant'`)
    - `amount` (DECIMAL(10,2)), `balance_after` (DECIMAL(10,2))
    - `description` (VARCHAR(255)), `created_at` (DATETIME)
9. `attendance` **(GPS Geofenced Punch Register):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `worker_id` (FK $\rightarrow$ `workers.id`)
    - `service_request_id` (FK $\rightarrow$ `service_requests.id`, NULLABLE)
    - `clock_in` (DATETIME), `clock_out` (DATETIME)
    - `clock_in_latitude` (DECIMAL(10,8)), `clock_in_longitude` (DECIMAL(11,8))
    - `is_verified_geofence` (BOOLEAN)
10. `contractor_reviews`:
    - `id` (PK, INT, AUTO_INCREMENT)
    - `customer_id` (FK $\rightarrow$ `customers.id`), `contractor_id` (FK $\rightarrow$ `contractors.id`), `service_request_id` (FK $\rightarrow$ `service_requests.id`)
    - `rating` (TINYINT: 1 to 5), `comment` (TEXT), `created_at` (DATETIME)
11. `worker_certifications` & `contractor_documents`:
    - Stores trade certificate numbers, issuing authorities (NCVT/ITI), validities, and verification document file paths.
12. `system_settings`, `activity_logs`, `support_tickets`, `notifications`:
    - Manages platform parameters (welfare percentages, floor wages), tamper-proof audit trails, user tickets, and WebSockets/DB alerts.

6. `quotations` **(Upfront Pricing & Materials):**

- `id` (PK, INT, AUTO_INCREMENT)
- `service_request_id` (FK $\rightarrow$ `service_requests.id`)
- `contractor_id` (FK $\rightarrow$ `contractors.id`)
- `labor_cost` (DECIMAL(10,2)), `material_cost` (DECIMAL(10,2))
- `gst_rate` (DECIMAL(5,2), DEFAULT 18.00), `discount` (DECIMAL(10,2))
- `total_amount` (DECIMAL(10,2))
- `status` (ENUM: `'draft'`, `'sent'`, `'approved'`, `'rejected'`)

7. `payments` **(Fair Wage Engine & Digital Signatures):**

- `id` (PK, INT, AUTO_INCREMENT)
- `service_request_id` (FK $\rightarrow$ `service_requests.id`)
- `invoice_number` (VARCHAR(50), UNIQUE)
- `total_amount` (DECIMAL(10,2))
- `worker_amount` (DECIMAL(10,2)) — 88% Living Wage
- `contractor_amount` (DECIMAL(10,2)) — 8% Society Operations
- `welfare_amount` (DECIMAL(10,2)) — 4% Suraksha Nidhi Fund
- `payment_method` (ENUM: `'cash'`, `'upi'`, `'card'`, `'bank_transfer'`)
- `payment_status` (ENUM: `'pending'`, `'paid'`, `'refunded'`)
- `signature_path` (VARCHAR(255)) — Base64 canvas e-signature file
- `paid_at` (DATETIME)

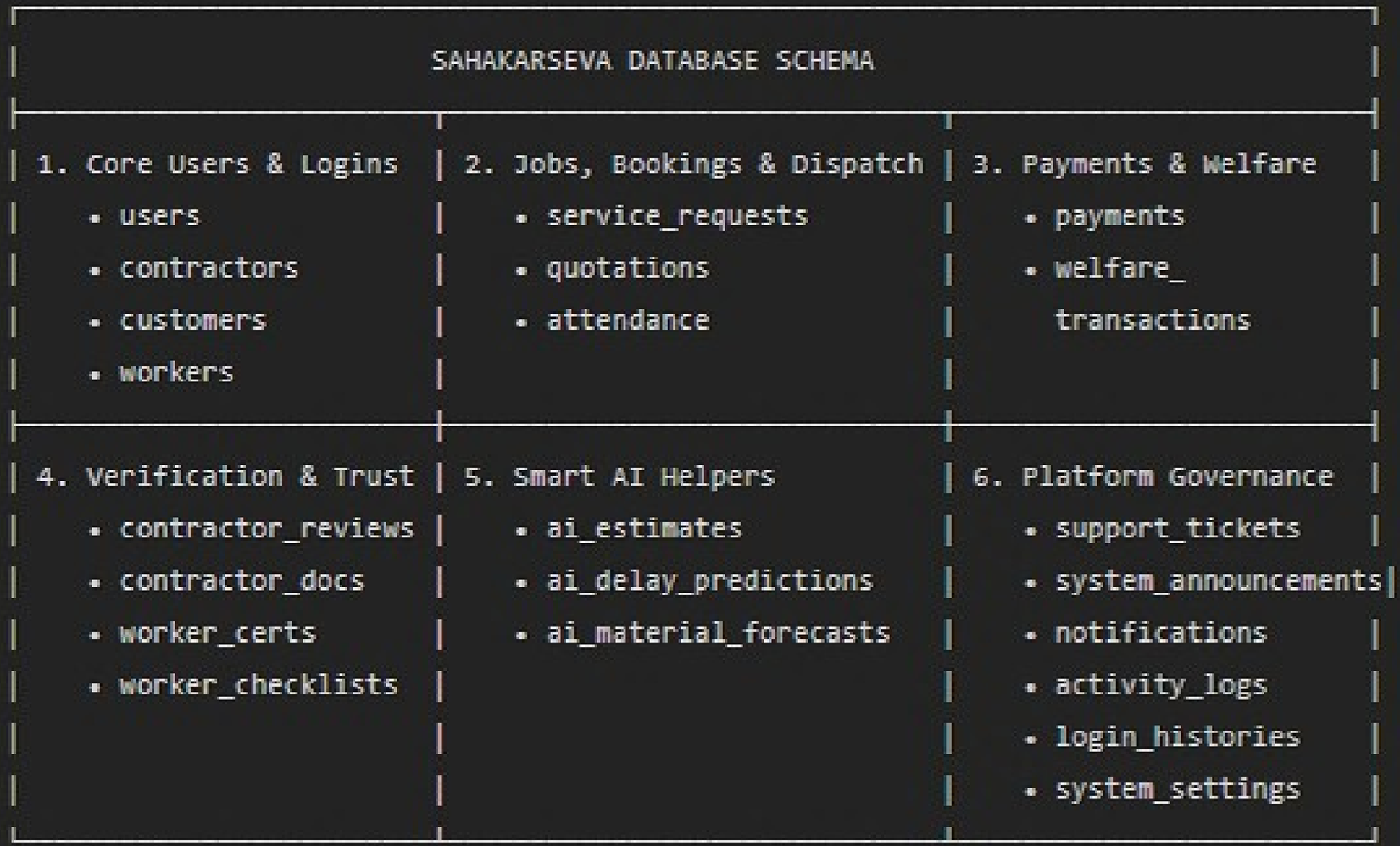
4. `workers` **(Certified Cooperative Artisans):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `user_id` (FK $\rightarrow$ `users.id`, UNIQUE)
    - `contractor_id` (FK $\rightarrow$ `contractors.id`, NOT NULL)
    - `skills` (VARCHAR(255)) — Comma-separated trades (e.g., "Electrician, Wiring")
    - `skill_tier` (ENUM: `'apprentice'`, `'skilled'`, `'master'`)
    - `hourly_rate` (DECIMAL(10,2)), `experience_years` (INT)
    - `current_latitude` (DECIMAL(10,8)), `current_longitude` (DECIMAL(11,8))
    - `is_available` (BOOLEAN, DEFAULT TRUE)
    - `pmsby_number` (VARCHAR(50)), `pmjjby_number` (VARCHAR(50)), `esic_number` (VARCHAR(50))
    - `welfare_fund_balance` (DECIMAL(10,2), DEFAULT 0.00)
5. `service_requests` **(The Core Job Lifecycle):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `tracking_number` (VARCHAR(30), UNIQUE) — e.g., `SR-2026-0042`
    - `customer_id` (FK $\rightarrow$ `customers.id`)
    - `contractor_id` (FK $\rightarrow$ `contractors.id`)
    - `worker_id` (FK $\rightarrow$ `workers.id`, NULLABLE)
    - `trade_category` (VARCHAR(50)) — e.g., "Plumbing", "Electrical"
    - `title` (VARCHAR(200)), `description` (TEXT), `service_address` (TEXT)
    - `latitude` (DECIMAL(10,8)), `longitude` (DECIMAL(11,8))
    - `urgency` (ENUM: `'normal'`, `'urgent'`, `'emergency'`)
    - `status` (ENUM: `'pending'`, `'assigned'`, `'in_transit'`, `'arrived'`, `'in_progress'`, `'completed'`, `'cancelled'`)
    - `arrival_otp` (VARCHAR(6)) — Cryptographic gate code
    - `audio_note_path` (VARCHAR(255)), `photo_path` (VARCHAR(255))
    - `scheduled_date` (DATETIME), `completed_at` (DATETIME)

### 4.1 Core Tables and Primary Fields

1. `users` **(Master Account Registry):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `username` (VARCHAR(80), UNIQUE, NOT NULL)
    - `email` (VARCHAR(120), UNIQUE, NOT NULL)
    - `password_hash` (VARCHAR(255), NOT NULL)
    - `role` (ENUM: `'publisher'`, `'admin'`, `'worker'`, `'customer'`)
    - `phone_number` (VARCHAR(20)), `address` (VARCHAR(255)), `profile_image` (VARCHAR(255))
    - `is_2fa_enabled` (BOOLEAN), `totp_secret` (VARCHAR(32))
    - `created_at` (DATETIME)
2. `contractors` **(Primary Labour Cooperative Societies):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `user_id` (FK $\rightarrow$ `users.id`, UNIQUE)
    - `name` (VARCHAR(150), NOT NULL) — Society Name
    - `registration_number` (VARCHAR(100), UNIQUE) — Cooperative Bye-Law Registration No.
    - `phone`, `email`, `address`, `city`, `service_area` (VARCHAR)
    - `is_verified` (BOOLEAN, DEFAULT FALSE) — Federation Approval Status
    - `rating` (FLOAT, DEFAULT 5.0), `total_reviews` (INT, DEFAULT 0)
3. `customers` **(Household & Housing Society Clients):**
    - `id` (PK, INT, AUTO_INCREMENT)
    - `user_id` (FK $\rightarrow$ `users.id`, UNIQUE)
    - `company_name` (VARCHAR(150)) — Apartment Complex / Housing Society Name
    - `billing_address` (TEXT), `gst_number` (VARCHAR(50))
    - `emergency_contact` (VARCHAR(20))

## 4. Database Schema & RDBMS Structure

The production database is optimized into **22 core tables** grouped into 6 logical clusters:



| | | | |
|---|---|---|---|
| **POST** | `/customer/invoice/sign` | Saves HTML5 canvas digital signature | Payload: `{"payment_id": int, "signature_data": base64}` |
| **GET** | `/admin/api/ai/recommend-workers` | AI ranking of artisans by proximity & skills | Query: `?request_id=<int>&trade=<str>` |
| **GET** | `/admin/api/ai/delay-predictions` | Forecasts delivery delays based on weather/load | Query: `?contractor_id=<int>` |
| **GET** | `/admin/api/ai/business-insights` | LLM-powered operational insights | Query: `?range=week |
| **POST** | `/admin/api/backup/create` | Generates a full system SQL database backup | Triggers backup snapshot engine |
| **GET** | `/worker/api/emergency-broadcasts` | Polls/fetches unassigned urgent jobs | Filter: By worker trade category and location radius |
| **POST** | `/worker/api/update-location` | Streams worker GPS coordinates | Payload: `{"latitude": float, "longitude": float}` |
| **POST** | `/worker/service-request/<id>/verify-arrival-otp` | Verifies customer OTP on-site | Path param: `job_id`; Payload: `{"otp": str}` |
| **POST** | `/publisher/security/force-logout-all` | Emergency session invalidation | Action: Broadcasts global logout signal |

Apex Federation / Super Admin ( `/publisher/...` )

- **Statewide Society Governance:** Review, verify, or suspend affiliated district labour cooperatives.
- **KYC & Legal Document Auditing:** Inspect uploaded registration certificates, PWD licenses, and public liability policies in a secure vault.
- **Dispute Arbitration Engine:** Impartial review and binding resolution of escalated customer-society disputes.
- **Emergency Security Controls:** Global platform maintenance mode switches, role-based lockouts, and instantaneous "Force Logout All Sessions" security signal.
- **State Circular Broadcasts:** Push official notices and safety guidelines to all registered workers and society managers.
- **Immutable Security Audit Log:** Real-time tamper-proof trail of administrative actions, logins, and IP addresses.

---

### 3.3 Primary REST API Endpoints

| HTTP Method | Route / Endpoint | Description | Query Filters / Parameters / Payload |
|---|---|---|---|
| **POST** | `/api/translate` | Batch multilingual translator proxy | Payload: `{"texts": [...], "target": "mr"` |
| **POST** | `/api/assistant` | Contextual AI conversational booking assistant | Payload: `{"message": str, "language": str}` |
| **GET** | `/api/search` | Global instant search across records | Query: `?q=<term>` (Searches jobs, workers, invoices) |
| **GET** | `/api/pdf/invoice/<int:id>` | Generates and streams PDF invoice | Path param: `payment_id` |
| **GET** | `/api/pdf/quotation/<int:id>` | Generates and streams PDF quotation | Path param: `quotation_id` |
| **GET** | `/api/pdf/report/<type>` | Generates and streams PDF business report | Path param: `financial` \| `workers` \| `operations` |
| **POST** | `/api/2fa/toggle` | Toggles TOTP Two-Factor Authentication | Payload: `{"enable": bool, "otp_code": str}` |
| **GET** | `/customer/api/geocode` | OpenStreetMap reverse geocoding proxy | Query: `?lat=<float>&lon=<float>` |
| **POST** | `/customer/book-service/submit` | Creates a new service booking | Form Data: `trade`, `urgency`, `lat`, `lon`, `audio_note`, etc. |
| **POST** | `/customer/booking/<id>/verify-otp` | Customer-side arrival confirmation | Path param: `booking_id`; Payload: `{"otp": "1234"}` |

### Customer Portal (`/customer/...`)

- **3-Step Service Booking Wizard:** Select from 10 skilled trades, input service details, record voice notes, and upload photo attachments of the issue.
- **Emergency 1-Click Dispatch:** Instant broadcast to all nearby verified artisans within a 7.5 km radius.
- **Live Job Tracking & Arrival Verification:** Track assigned technician status (`Requested` $\rightarrow$ `Assigned` $\rightarrow$ `In-Transit` $\rightarrow$ `Arrived`). Provides the customer with a private arrival OTP.
- **Pre-Dispatch Booking Modification:** Edit address, time slot, or cancel bookings prior to technician dispatch.
- **Quotation Review & Digital Sign-off:** Review itemized material and labor quotes; approve or reject online.
- **Digital Invoicing & Signature:** Draw digital signatures directly on an HTML5 canvas pad before settling bills via UPI/Cash/Card; download stamped PDF receipts.
- **Rating & Feedback Mechanism:** 1-to-5 star ratings with qualitative feedback updating public society trust scores.
- **Help Desk & Dispute Escalation:** Raise support tickets or escalate disputes to the State Federation.

### Worker / Artisan Portal (`/worker/...`)

- **Live Duty & Availability Toggle:** Switch between "Available" and "Busy" status; updates live GPS location coordinates.
- **Emergency Broadcast Radar:** Receive real-time audio and visual alerts via WebSockets when emergency jobs are requested nearby.
- **Job Execution Flow:** One-click job claiming, transition status to "In-Transit", prompt for customer arrival OTP, follow mandatory safety checklists, and take completion photos.
- **GPS & QR Attendance:** Punch in/out using device geolocation geofences and customer QR scans.
- **Earnings & Payslip Hub:** Track earnings with detailed breakdowns (88% living wage, 8% society charge, 4% Suraksha Nidhi); download generated payslips.
- **Suraksha Nidhi Passbook (`/worker/benefits`):** View active social security insurance numbers (PMSBY, PMJJBY, ESIC) and track auto-credited welfare balances.

---

## 3.2 Admin Panel Features

### Cooperative Society Admin (`/admin/...`)

- **Operations Dashboard:** Live KPI metrics for active service requests, available artisans, monthly turnover, and attendance.
- **Artisan Profiling & Verification:** Onboard technicians, assign skill tiers (*Apprentice*, *Skilled Craftsman*, *Master Artisan*), and inspect ITI/NCVT trade diplomas.
- **Dispatch Management:** Manual or AI-assisted technician assignment using Haversine GPS proximity and workload scores.
- **AI Demand & Workforce Allocation (`/admin/ai-demand`):** 7-day rolling trade demand forecasts with predictive surge alerts across municipal wards.
- **Financial Ledger & Billing:** Generate itemized quotations, calculate 18% GST, track collections, and audit statutory fee splits.
- **Automated Database Backups:** One-click SQL backup creation, download, and disaster recovery snapshotting.

| | | |
|---|---|---|
| | Flask-SQLAlchemy 3.1+ & SQLAlchemy 2.0 ORM | Object-relational mapping, connection pooling, and transactional data access. |
| | Flask-SocketIO 5.3+ (WebSockets) | Real-time bi-directional emergency job dispatch and live tracking alerts. |
| **Database** | MySQL 8.0 / MariaDB (via `mysql-connector-python`) | Relational database engine supporting multi-tenant isolation, foreign key constraints, and transactional consistency. |
| **Security & Auth** | PyOTP 2.9+ & qrcode 8.0+ | Time-based One-Time Password (TOTP) Two-Factor Authentication (2FA) and QR codes. |
| | Werkzeug Security | Salted PBKDF2/SHA-256 password hashing and session tokens. |
| **AI & External APIs** | Google Gemini 2.0 (`gemini-2.0-flash`) & Grok (xAI) API | Generative scope breakdown, AI cost estimations, and intelligent trade demand predictions. |
| | OpenStreetMap Nominatim Proxy | Reverse geocoding of GPS coordinates to human-readable street addresses. |
| | Google Translate API Batch Proxy | Server-side translation proxy (`/api/translate`) supporting English, Marathi (मराठी), and Hindi (हिंदी). |
| **Document Engine** | ReportLab 4.0+ | Automated server-side generation of downloadable PDF invoices, quotations, and payslips. |

## 3. Core Features & System Breakdown

The system enforces strict **Role-Based Access Control (RBAC)** across four primary user personas: **Apex Federation (Publisher)**, **Cooperative Society Admin**, **Skilled Member Artisan (Worker)**, and **Household / Housing Society Customer**.

### 3.1 Public & User Features

#### Public Portal (Visitors)

- **Marketplace Discovery (`/marketplace`, `/home`):** Browse verified district cooperative societies, view certified trades, and inspect public ratings.
- **Cooperative Society Onboarding (`/register-contractor`):** Self-service digital application with bye-law certificate uploads, registration number verification, and operational district mapping.
- **Citizen Registration (`/register`):** Fast sign-up for household clients with phone and address details.

#### Customer Portal (`/customer/...`)

4. **Physical Verification, Anti-Fraud & Arrival Security:**
   - Eliminates remote job falsification through a **Cryptographic 4-Digit/6-Digit Arrival OTP Gate** generated on the customer's device, requiring physical technician entry before work timers unlock.
   - Geofenced GPS attendance punch-in with QR verification guarantees physical presence.
5. **Transparency & Trust in Pricing:**
   - Removes hidden surge pricing and unverified contractor rates through AI-powered upfront cost estimation and pre-approved cooperative trade tariffs across 10 skilled trades.

---

## 2. Type of Application & Tech Stack

### Application Category

- **Multi-Tenant Responsive Web Application** with Progressive Web App (PWA) capabilities (`service-worker.js`), optimized for smartphones, tablets, and desktop workstations.
- Features a dedicated mobile-first design with proportional touch targets (29px–32px inputs), auto-wrapping card grids, audio recording inputs, and collapsible navigation.

### Complete Technology Stack

| Layer | Technologies & Frameworks | Description / Purpose |
|---|---|---|
| **Frontend** | HTML5, Vanilla CSS3 (Custom Glassmorphism), JavaScript (ES6+), Jinja2 | Responsive UI rendering, CSS variables, media queries (`max-width: 768px/576px`), zero dependency bloat. |
| | HTML5 Canvas API | Touch and mouse e-signature pad for legally binding invoice sign-offs. |
| | MediaRecorder API | In-browser audio note recording for customer problem descriptions and field notes. |
| | Font Awesome 6.4 & Google Fonts (Inter) | Typography and consistent icon sets. |
| **Backend** | Python 3.10+ & Flask 3.0+ | Core application logic, routing, middleware, and multi-tenant row scoping. |
| | Flask-Login 0.6+ | Session-based user authentication and role-based access control (RBAC). |

Act as a Technical Documentation Specialist. I am preparing a formal Field Project Report for my college submission based on this project.

Please analyze this repository / project and provide a detailed summary covering the following sections in clear, structured text:

1. Project Overview & Business Requirements:

Worked for 3m ›

# Project Field Report: SahakarSeva Cooperative Services Platform

## 1. Project Overview & Business Requirements

### Project Title & Domain

- **Official Project Title: SahakarSeva** — *Cooperative Gig Services Platform for Household & Community Services*
- **National Problem Statement ID: 26089** (Ministry of Cooperation, Govt. of India / National Council for Cooperative Training – NCCT)
- **Domain / Industry:** Multi-Tenant SaaS, Cooperative Gig Economy, Civic Tech, Social Security, and On-Demand Home Services.

### Real-World Problem & Business Processes Addressed

1. **Dismantling Private Aggregator Exploitation:**
   - Commercial gig platforms (Urban Company, TaskRabbit, etc.) levy aggressive commission cuts of **20% to 35%** on independent blue-collar service workers while categorizing them as independent contractors with zero safety nets.
   - **SahakarSeva** implements a statutory cooperative fair-wage model: **88%** of labor charges go directly to the worker's bank account, **8%** funds primary cooperative society tooling and field operations, and **4%** is credited to the **Suraksha Nidhi** worker welfare and insurance fund.
2. **Digital Empowerment of Labour Cooperative Societies:**
   - Primary Labour Cooperative Societies (PACS / Labour Cooperatives) possess pools of certified artisans (electricians, plumbers, carpenters, masons, painters) who lack digital booking tools and market visibility.
   - The system provides a centralized digital marketplace enabling district-level cooperative societies to register, verify member credentials, dispatch workers, and issue standardized, GST-compliant invoices.
3. **Institutionalizing Social Security & Welfare for Unorganized Labour:**
   - Connects artisan profiles directly with government-backed insurance schemes (**Pradhan Mantri Suraksha Bima Yojana - PMSBY**, **Pradhan Mantri Jeevan Jyoti Bima Yojana - PMJJBY**, and **ESIC**).
   - Automatically debits annual policy premiums (₹20 for PMSBY, ₹436 for PMJJBY) directly from accumulated job cess in the *Suraksha Nidhi* passbook.
4. **Physical Verification, Anti-Fraud & Arrival Security:**

SMART INDIA HACKATHON 2026

## PROBLEM STATEMENT DETAILS

| | |
|---|---|
| **Problem Statement ID** | 26089 |
| **Problem Statement Title** | Cooperative Gig Services Platform for Household & Community Services |
| **Description** | · Background Labour Cooperative Federations and Labour Cooperative Societies possess a large pool of skilled workers such as electricians, plumbers, carpenters, painters, domestic helpers, caregivers, drivers, gardeners,cleaners, and technicians. However, they lack a structured digital platform to connect these workers with households and institutions requiring such services.Private platforms currently dominate this market, while cooperative workers often remain underutilized despite having skills and local presence.<br>· Problem Statement To develop a cooperative-owned digital service marketplace platform that enables Labour Cooperative Federations and Labour Cooperative Societies to provide verified household and community services while ensuring fair wages, worker welfare, and consumer trust.<br>· Expected Solution Features |
| **Organization** | Ministry of Cooperation |
| **Department** | National Council for Cooperative Training (NCCT) |
| **Category** | Software |
| **Theme** | Agriculture, FoodTech & Rural Development |
| **Youtube Link** | |
| **Dataset Link** | |
| **Contact info** | |

### Member 2: Frontend Engineering, PWA & Responsive UX

- **Module Ownership:** `templates/`, `static/css/style.css`, `static/js/translations.js`, `service-worker.js`.
- **Key Deliverables:**
  - Created the responsive mobile-first UI using vanilla CSS variables, responsive typography, and 29px/32px touch targets.
  - Built the 3-step booking wizard with HTML5 audio recording (`MediaRecorder`) and camera photo attachments.
  - Implemented the client-side multilingual translation engine using Google Translate batch proxying with lossless WeakMap DOM caching.
  - Configured service worker caching for offline access and PWA installation.

### Member 3: AI Engine, GIS Geolocation & Real-Time Dispatch

- **Module Ownership:** `services/ai_service.py`, `services/dispatch_service.py`, `routes/worker.py`, `models/attendance.py`.
- **Key Deliverables:**
  - Integrated Google Gemini 2.0 / Grok APIs for automated job scope breakdowns and cost estimations.
  - Developed the multi-factor smart artisan recommendation algorithm based on NLP trade keywords, active job loads, and Haversine proximity.
  - Built the OpenStreetMap Nominatim reverse geocoding proxy and GPS geofenced attendance tracking.
  - Configured Flask-SocketIO WebSockets channels for instantaneous emergency broadcast notifications and live tracking.

### Member 4: Payments, Statutory Welfare & Administrative Governance

- **Module Ownership:** `services/welfare_service.py`, `services/pdf_service.py`, `routes/publisher.py`, `routes/customer.py`, `models/finance.py`.
- **Key Deliverables:**
  - Designed the statutory fair wage engine with automatic 88% / 8% / 4% payment distribution.
  - Integrated the *Suraksha Nidhi* passbook with tracking for government social security schemes (PMSBY, PMJJBY, and ESIC).
  - Built the HTML5 canvas touch e-signature component and ReportLab PDF invoice/payslip generation pipeline.
  - Developed the Apex Federation Governance console, dispute arbitration desk, and emergency platform security lockouts.

## 6. Member Contributions & Module Breakdown

For academic submission and field project evaluation, the engineering workload of the repository is modularly distributed among team members:

| PROJECT TEAM MODULE BREAKDOWN | |
|---|---|
| Team Member 1 | Backend Architecture, Database & Security |
| Team Member 2 | Frontend Engineering, PWA & UX Design |
| Team Member 3 | AI Engine, GIS Geocoding & Emergency Dispatch |
| Team Member 4 | Digital Payments, Statutory Welfare & Governance |

### Member 1: Backend Architecture, Database & Security

- **Module Ownership:** `app.py`, `database.py`, `config.py`, `routes/auth.py`, `models/user.py`, `models/contractor.py`.
- **Key Deliverables:**
  - Configured MySQL database engine, connection pooling, and multi-tenant row isolation (`contractor_id`).
  - Engineered user authentication system with Werkzeug password hashing, session management, and RBAC guards.
  - Implemented Time-based One-Time Password (TOTP) Two-Factor Authentication using PyOTP and QR code generation.
  - Built database backup creation and export routines (`export_db.py`, `seed_db.py`).